

Chapter 12 - Hibernate Relationship Mapping

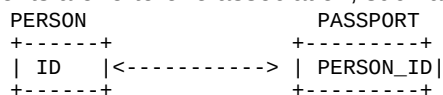
1. Introduction

Relationship mapping defines how entities are associated with each other. Hibernate supports one-to-one, one-to-many, many-to-one and many-to-many relationships.

Annotation	Relationship
@OneToOne	One entity maps to one entity
@OneToMany	One parent maps to many children
@ManyToOne	Many child records map to one parent
@ManyToMany	Many records map to many records
@JoinColumn	Defines the foreign key column
mappedBy	Specifies the inverse (non-owning) side

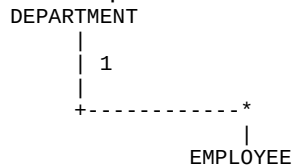
2. @OneToOne

Represents a one-to-one association, such as Person ↔ Passport.



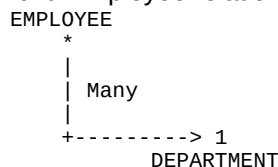
3. @OneToMany

Represents one parent associated with many child entities, such as Department → Employees.



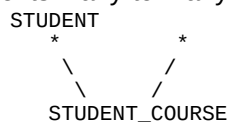
4. @ManyToOne

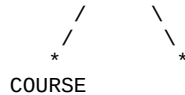
Represents many child entities associated with one parent. This is the owning side in a typical Department–Employee relationship.



5. @ManyToMany

Represents many-to-many associations using a join table, such as Student ↔ Course.





6. @JoinColumn

@JoinColumn specifies the foreign key column on the owning side of the relationship. Example: @JoinColumn(name="DEPT_ID").

7. mappedBy

mappedBy identifies the inverse side of a bidirectional relationship. The owning side manages the foreign key.

8. Unidirectional vs Bidirectional

Unidirectional: navigation is possible in one direction only. Bidirectional: both entities reference each other. The owning side updates the relationship in the database.

9. Fetch & Cascade

Relationships can use FetchType.LAZY or FetchType.EAGER. Cascade options (PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL) control propagation of operations.

10. Example

```
@Entity
class Department {

    @OneToMany(mappedBy="department")
    private List<Employee> employees;
}

@Entity
class Employee {

    @ManyToOne
    @JoinColumn(name="DEPT_ID")
    private Department department;
}
```

11. Best Practices

- Prefer LAZY fetching for collections.
- Define cascade types carefully.
- Keep one owning side using mappedBy correctly.
- Avoid unnecessary bidirectional relationships.

12. Interview Questions

1. Difference between @OneToMany and @ManyToOne? 2. What is the owning side? 3. What does mappedBy do? 4. Why use @JoinColumn? 5. Unidirectional vs Bidirectional?